4b. pthreads 101

0. Threads are fun!

EXPLANATION:

Processes (processes.c):

● Background:

- We have a global variable x=2.
- The program creates a child process using fork().
- The child process increments x by 5.
- Both the parent and the child processes prints the value of x.

● **Output** :

- PID of Process: 4128215
- Value of x: 2
- PID of Process: 4128216
- Value of x: 7

Explanation:

■ Parent Process (PID: 4128215): It prints the initial value of x, which is 2.

■ Child Process (PID: 4128216): After the child increments x by 5, it prints the new value of x, which is 7.

● **Reasoning:**

- **Separate Memory**: When a process is forked, the parent and child processes have separate memory spaces. Changes in one process don't affect the other. That's why the parent's x stays 2, while the child's x becomes 7.

Threads (threads.c):

● **Background**:

- We have a program that starts with a global variable x=2.
- The program creates two threads.

- One thread increments x by 5.
- Both threads print the value of x.

● **Output** :

- PID of Foo: 4128512
- PthreadID : 132609988363968
- Value of x: 7
- PID of Bar: 4128512
- PthreadID : 132609977878208
- Value of x: 7

Explanation:

- Thread Foo: It increments x by 5 and then prints the value of x, which is now 7.
- Thread Bar: It prints the value of x after Foo has already incremented it, so it also prints 7.

● **Reasoning**:

- **Shared Memory**: Threads within the same process share the same memory space. When one thread changes a variable, the other threads see the change. That's why both threads print the same value of x, which is 7.

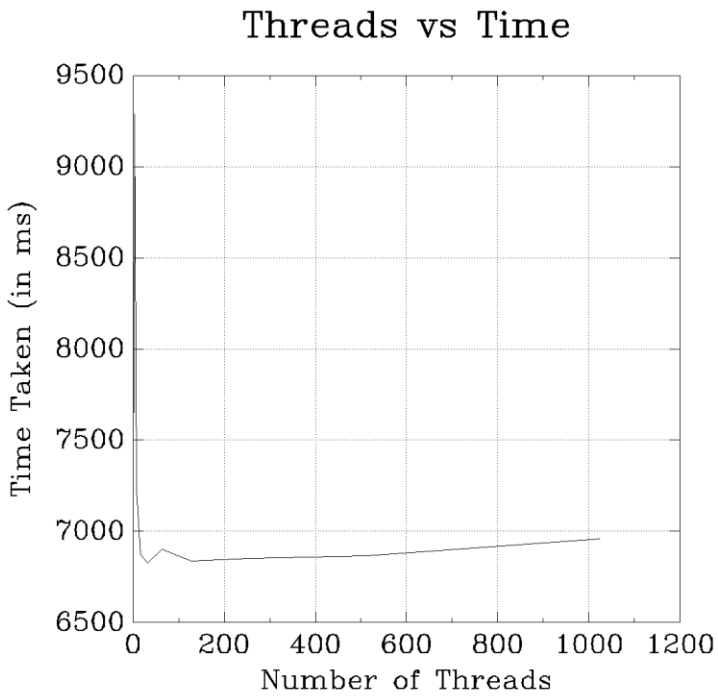
Conclusion:

- **Processes**: Separate memory spaces mean changes in one process don't affect another. Parent prints 2, child prints 7.
- **Threads**: Shared memory space means changes by one thread are seen by all. Both threads print 7.

1. Where did the money go?

Task 1B: The Sweet Spot

Performance Analysis of Multi-threaded Program:



Result:

Number of Threads Time Taken (µs)

Number of Threads	Time Taken (µs)
2	7653.190
4	9287.533
8	7232.004
16	6872.361
32	6827.329
64	6901.133
128	6837.150
256	6852.229
512	6867.810
1024	6957.815

Explanation:

As the number of threads increased from 2 to 4, the execution time increased. This suggests that the system is spending more time managing the threads, rather than benefiting from parallelism.

The time taken was lowest at 32 threads, meaning that adding more threads didn't help performance and actually made it worse. This indicates that the overhead of managing so many threads became significant.

After 32 threads, the execution time fluctuated slightly but did not show major improvement. The time did not decrease significantly even when 512 threads were used, showing diminishing returns where additional threads either don't improve performance or even slow the program down due to the extra overhead.

The analysis indicates that increasing the number of threads does not necessarily improve performance. As the number of threads grows, the system has to allocate more resources to manage thread switching, memory sharing, and synchronization, leading to performance degradation. Thus, in multi-threaded applications, it's important to find the right balance between parallelism and overhead. Blindly increasing the number of threads without considering the workload and system limits can result in poor performance.